



Факултет по математика и информатика
Софийски университет "Св. Климент Охридски"

НЕВРОНЕН МАШИНЕН ПРЕВОД ЧРЕЗ ГЕНЕРАТИВЕН ЕЗИКОВ МОДЕЛ

КУРСОВ ПРОЕКТ ПО "ТЪРСЕНЕ И ИЗВЛИЧАНЕ НА ИНФОРМАЦИЯ. ПРИЛОЖЕНИЕ НА ДЪЛБОКО МАШИННО
ОБУЧЕНИЕ"

Делян Бойчев
Спец. Информатика
Ф.№ 3М10400338
dlboychev@uni-sofia.bg

1 Въведение¹

Целта на курсовия проект е разработката и имплементацията на система за невронен машинен превод (NMT) от български към английски език. Задачата се свежда до моделиране на условната вероятност на целевата поредица (английски текст) при дадена изходна поредица (български текст). Разглеждат се проблемите, свързани с избор на архитектура, токенизация (tokenization) и оптимизация на модела.

Стандартният подход за създаване на речник за задача на превод е разделяне по думи, но този метод всъщност се оказва неефективен според Sennrich et al. [1]. Проблемите, които възникват са **размерът на речника, работа на модела с непознати думи**, като при задачи за превод се появява проблемът и с нееднозначността и понякога директното представяне на думите да бъдат токени не води до добри резултати. Sennrich et al. [1] предлагат Byte-Pair Encoding (BPE) алгоритъм за създаване на речника от поддуми, което решава тези проблеми.

При избора на архитектура бяха разгледани традиционните рекурентни невронни мрежи (например LSTM [2]), които обаче имат съществени недостатъци.

Липса на паралелизъм Последователната обработка прави обучението бавно и неефективно.

Загуба на контекст Трудно запаметяват връзки между отдалечени думи в дълги изречения поради потока от градиенти към началото.

Bottleneck Представянето на цялото изречение с един вектор води до загуба на информация.

За решаването на задачата беше избрано използването на BPE [1] алгоритъмът за създаване на речник от поддуми и архитектурата Transformer [3] за реализацията на модела. Тя заменя рекурентността с attention механизъм, позволявайки паралелно обучение и директно улавяне на зависимости между всички думи, независимо от разстоянието им. Това я прави дефакто стандарт за съвременни NLP задачи.

2 Методология

2.1 Токенизация

За токенизация се използва BPE алгоритмът, показан в Алгоритъм 1. Това е опростеният и неоптимизиран вариант на алгоритъма за създаване на правила за сливане. В реализираната имплементация

¹Тежестите се намират [на този линк](#).

пресмятането на честотите всеки път е заменено с едно пресмятане в началото и обновяване по време на изпълнение. За намиране на максимален елемент е използвана класическата приоритетна опашка на база двоичен Heap със сложност $O(\log n)$ за добавяне и $O(1)$ за получаване на максимален елемент, заменяйки търсенето в речник със сложност $O(n)$.

При кодиране на дума се следват правилата за сливане от наредения списък и се прилагат докато вече няма правила за тази дума и се повтаря докато не се стигне края на изречението. Накрая всеки токен се представя като индекс от речника с токени.

Съответно при декодиране отново по индекс се взема символното представяне на токена, слепва се дума по дума (където има маркер, че токенът не е край на думата) докато не се достигне края на думата. Повтаря се за всяка дума до края на изречението.

За съставния речник са направени $K = 30000$ сливания.

Algorithm 1 BPE алгоритъм, правила за сливания

Require: Корпус C (списък от изречения), брой сливания K

Ensure: Списък от сливания M

```

1:  $V \leftarrow$  честоти на думите в  $C$  ▷ речник: дума  $\rightarrow$  честота
2: Всяка дума  $w$  във  $V$  се преобразува до символи  $[c_1, \dots, c_n, \langle w \rangle]$ 
3:  $M \leftarrow []$  ▷ Нареден списък от правила за сливане
4: for  $t \leftarrow 1$  to  $K$  do
5:    $P \leftarrow$  празен брояч ▷ двойка  $(x, y) \rightarrow$  честота
6:   for all  $(word, freq) \in V$  do
7:     for  $i \leftarrow 1$  to  $|word| - 1$  do
8:        $P[(word[i], word[i + 1])] \leftarrow P[(word[i], word[i + 1])] + freq$ 
9:     end for
10:  end for
11:  if  $P$  е празен then
12:    break
13:  end if
14:   $best \leftarrow$  най-честата двойка в  $P$  ▷ при равенство се сравнява лексикографски
15:  append  $best$  to  $M$  ▷ Във всяка дума се заменят всички срещания на  $best = (a, b)$  с токен  $ab$ 
16:   $V' \leftarrow$  празен речник
17:  for all  $(word, freq) \in V$  do
18:     $new \leftarrow word$  след сливане на всички срещания на  $best$ 
19:     $V'[new] \leftarrow V'[new] + freq$  ▷ ако  $new$  липсва, се приема  $V'[new] = 0$  (рядко има повторение след сливане)
20:  end for
21:   $V \leftarrow V'$ 
22: end for
23: return  $M$ 

```

2.2 Архитектура

Имплементира се Transformer архитектура [3], използваща така наречения Masked Multi-Head Attention механизъм, представен във Фигура 1.

Той се състои от Query (Q), Key (K) и Value (V). След трансформиране на скритото състояние (hidden state) $x \in \mathbb{R}^{n \times d_{model}}$ с линеен слой, то се разбива на n_{heads} . Прилагат се линейните трансформации и се пресмята произведението на двете матрици, съпоставяйки всяко векторно представяне на токен с всяко (идеята идва от скаларното произведение, което се използва като метрика за подобие на вектори). Замаскират се стойностите чрез поелементна сума на тази матрица с триъгълна матрица с нули в долната част и $-\infty$ в горната част, за да може да не изтичат съпоставянията на токени след текущия (тъй като се предсказва само следващия токен). Има нужда от скалиране на тези стойности в матрицата, за да не растат много и се дели поелементно на $\sqrt{d_k}$. Прилага се $softmax(\cdot)$ по редовете на получената квадратна матрица, което дава attention scores матрица с нормализирани стойности за съпоставянията по редове. Съответно нейната сума по редове е 1. Като се умножи за всички глави $softmax(\frac{Q_i K_i^t}{d_k}) V_i, i \in \{1, \dots, n_{heads}\}$ се дава различна "важност" на токениите. Тя е адаптивна зависимост

от контекста и моделът се учи как да оценява токените. Накрая се конкатенират в същия ред, в който са разделени главите, и се прилага линейна трансформация на новия hidden state.

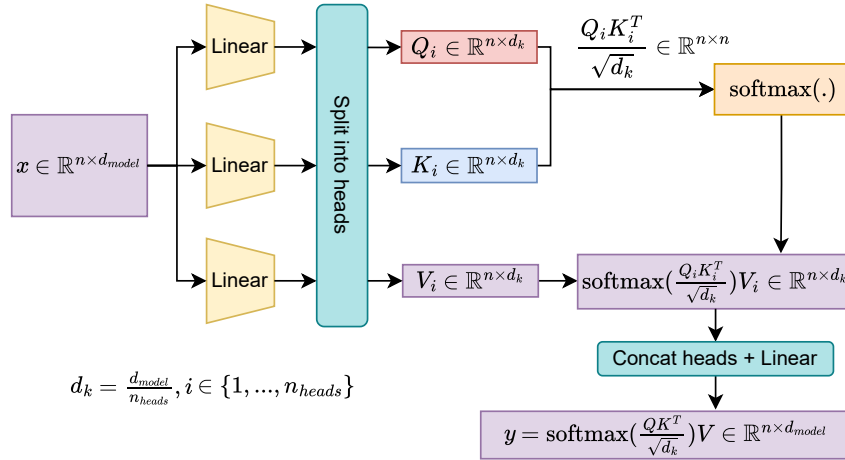
Чрез този механизъм се сглобява цялата архитектура. Всеки слой се състои от Attention, LayerNorm [4] и съответно Feed Forward (Multi-Layer Perceptron - два линейни слоя с активационна функция ReLU между тях).

Цялата архитектура се състои от Embedding слой, който мапва всеки токен към learnable вектор за него. Към тях се добавят позиционни представяния за всеки токен. Следват няколко Transformer слоя и накрая има линеен слой, който за всеки токен от поредицата връща вероятности за всеки токен от речника.

В конкретния случай беше установено, че броят слоеве $n_{layers} = 3$, размерността на скритите състояния $d_{model} = 512$ и броят на скритите състояния $n_{heads} = 8$ са достатъчни за задачата. Размерността на Feed Forward е заимствана от Sennrich et al. [1], където тя е $f_{dim} = 4d_{model}$. Между различните слоеве, включително и в Attention механизма е добавен Dropout, който занулява стойностите, за да се намали overfit ("пренапасване" върху данните за трениране). В реализацията е с вероятност $p = 0.1$. Разглеждат се и параметрите, представени от Araabi and Monz [5], но след експеримент се оказва, че $n_{layers} = 6$ е прекалено много и моделът започва да overfit-ва.

Допълнително, за да се забърза скоростта на генериране за модела, се имплементира методът *KV Caching* [6], който използва K и V за всички токени освен новогенерирания.

За генериране на крайния превод се използва декодиране чрез алгоритъма за търсене по лъчи (*Beam Search*) [7]. При него се поддържат паралелно фиксиран брой най-вероятни хипотези на всяка стъпка. Оценката на всяка хипотеза се формира със сума на база логаритмичните вероятности на предсказаните токени. Тъй като тази сума би намалила оценката за по-кратки изречения, се прилага нормализация спрямо дължината на генерираната последователност. Това балансира резултата и предотвратява преждевременното спиране на превода. Допълнително се налагат ограничения, които изключват възможността за генериране на специални токени. Тук прилагаме *KV Caching* динамично в зависимост от избраното продължение.



Фигура 1: Multi-Head Attention механизъм

2.3 Оптимизация

Използва се оптимизаторът Adam [8], който комбинира momentum и адаптивно скалиране на скоростта на обучение (learning rate) за всеки параметър чрез оценки на очакваната стойност и дисперсията на градиента. Това води до по-стабилно и бързо обучение в сравнение със стандартния Stochastic Gradient Descent (SGD), особено при шумни градиенти. Параметрите използват стандартни стойности - $\beta_1 = 0.9$, $\beta_2 = 0.999$. Добавя се към имплементацията и Learning Rate Scheduler с линейно повишаване в началото (warmup) - подобно като при Vaswani et al. [3] и спускане по косинусовата крива (cosine annealing) [9] до края на обучението. В началото, когато моделът има случайни тежести и градиентите имат нестабилни норми (показващи големината на стъпката), се предотвратява рискът от големи осцилации и разходимост. Прилага се Gradient Clipping [10] стратегия, която скалира големината на градиента, но не променя посоката му.

2.4 Оценяване

Моделите са оценени с метриците перплексия (Perplexity) и BLEU [11], които са стандарт за оценка на машинен превод. **Перплексията** може да се представи като $e^{\mathcal{L}_{CE}}$, където $\mathcal{L}_{CE}$ е кръстосана ентропия (Cross-Entropy) с основа на логаритъма e . Тази метрика може да се разгледа като среден брой възможни токени за всяка стъпка. Колкото по-малко са, толкова по-голямо ниво на увереност показва моделът. **BLEU** метриката от своя страна проверява за всяка последователност от n думи (n -грами) в истинския превод и генерирания, използва техния брой (който обаче за дадена n -грама е ограничен до този, който присъства в референция). В конкретния случай $n = 4$ и се изчислява съответно BLEU-4.

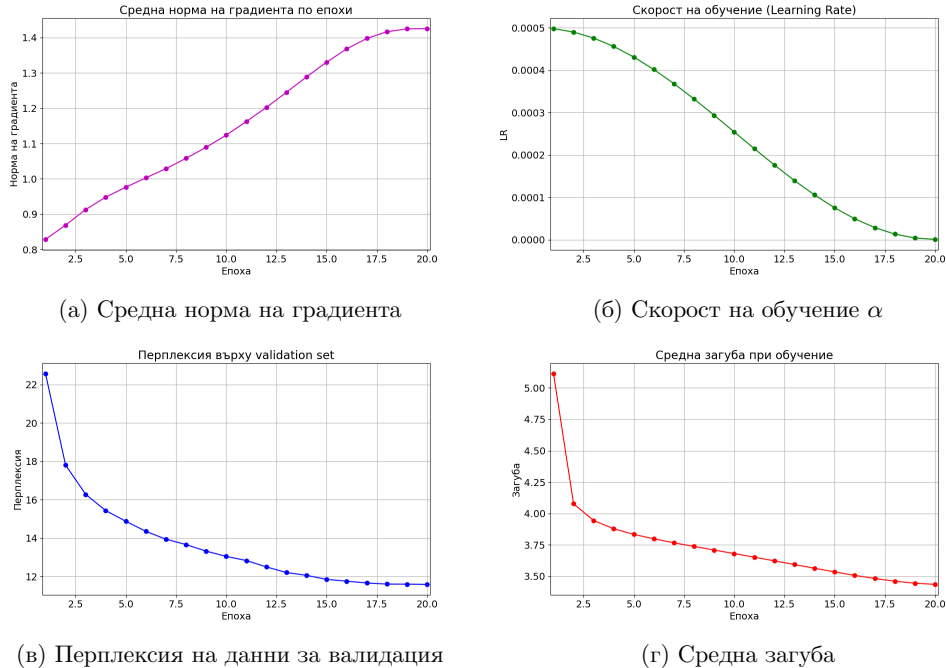
3 Експерименти

Фиксирани са следните експерименти в този ред, за да се установи: i) правилно ли е имплементирана архитектурата ii) кои параметри подпомагат за по-добра производителност. За всички експерименти са приложени скорост на обучение $\alpha = 0.0005$, параметър за затихване на теглата (weight decay) $\lambda = 0.0001$, gradient clipping праг $g_c = 1$, dropout вероятност $p = 0.1$, брой епохи $T = 20$ и посявка (random seed) $s = 42$.

- Първоначално трениране на малък модел с $n_{layers} = 1, d_{model} = 512, n_{heads} = 2$
- Трениране на голям модел с $n_{layers} = 6, d_{model} = 512, n_{heads} = 8$
- Трениране на половина по-малък модел с $n_{layers} = 3, d_{model} = 512, n_{heads} = 8$
- Добавяне на label smoothing [12] параметър

След изпълнението на експериментите се оказва, че големият модел с $n_{layers} = 6$ overfit-ва на данните и получава по-нисък BLEU score, същевременно му трябва и повече епохи за оптимизация. Променя се $n_{layers} = 3$ и се изпълняват следващите експерименти. Разработките на Agaabi and Monz [5] и Müller et al. [13] показват, че техниката за label smoothing помага при задача за машинен превод. Допълнително Müller et al. [13] показват подобрение до 0.5 в BLEU оценката с label smoothing параметър $\alpha_1 = 0.1$. Изпълнен е експеримент, който потвърждава повишаването на BLEU с 0.5, но се увеличава и перплексията с 0.5. Тъй като крайната оценка BLEU е важна за това колко плавен е изказът в едно изречение, се избира този модел като най-добър.

4 Резултати



Фигура 2: Графики по време на трениране по епохи

Във Фигура 2 са представени най-важните параметри, които се следят по време на обучението, които гарантират сигурност, че то протича нормално.

Първо, от Фигура 2а може да се отбележи, че моделът не изпитва зануляване или експлодиране на градиенти - тежестите се променят с нормална стъпка. Стъпката се контролира и от параметъра за скорост на обучение (Фигура 2б), който намалява по косинусовата крива - не се забелязва постепенното увеличаване в началото, защото се случва през цялата първа епоха. От Фигура 2г се заключава, че загубата схожда нормално, не се забелязва повишаване на средната загуба за епоха никъде. Фигура 2в показва, че сходимостта се случва някъде около средата на 17-та епоха, защото вече перплексията почти не намалява.

Таблица 1: Резултати на модела

Модел	BLEU-4	Perplexity
Greedy	41.08	12.26
Beam search ($w = 4$)	42.57	12.26

След провеждане на тестване на модела се получават резултатите, показани в Таблица 1, които индират, че моделът има високо ниво на увереност - средно 12 опции за предсказване на токен са равно вероятни. Търсенето по лъчи подобрява BLEU-4 с повече от 3.5%. Разбира се перплексията остава същата, защото тя се смята на база загубата и токен по токен. Стойността на BLEU-4 потвърждава силно съвпадение с истинските преводи при използваната процедура за оценка, която при други данни, но на същата задача, се постига от съвременните модели [14].

Литература

- [1] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units, 2016. URL <https://arxiv.org/abs/1508.07909>.
- [2] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Comput.*, 9(8):1735–1780, November 1997. ISSN 0899-7667. doi: 10.1162/neco.1997.9.8.1735. URL <https://doi.org/10.1162/neco.1997.9.8.1735>.
- [3] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023. URL <https://arxiv.org/abs/1706.03762>.
- [4] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016. URL <https://arxiv.org/abs/1607.06450>.
- [5] Ali Araabi and Christof Monz. Optimizing transformer for low-resource neural machine translation, 2020. URL <https://arxiv.org/abs/2011.02266>.
- [6] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference, 2022. URL <https://arxiv.org/abs/2211.05102>.
- [7] Markus Freitag and Yaser Al-Onaizan. Beam search strategies for neural machine translation. In Thang Luong, Alexandra Birch, Graham Neubig, and Andrew Finch, editors, *Proceedings of the First Workshop on Neural Machine Translation*, pages 56–60, Vancouver, August 2017. Association for Computational Linguistics. doi: 10.18653/v1/W17-3207. URL <https://aclanthology.org/W17-3207/>.
- [8] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017. URL <https://arxiv.org/abs/1412.6980>.
- [9] Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts, 2017. URL <https://arxiv.org/abs/1608.03983>.
- [10] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks, 2013. URL <https://arxiv.org/abs/1211.5063>.
- [11] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: a method for automatic evaluation of machine translation. In Pierre Isabelle, Eugene Charniak, and Dekang Lin, editors, *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, Philadelphia, Pennsylvania, USA, July 2002. Association for Computational Linguistics. doi: 10.3115/1073083.1073135. URL <https://aclanthology.org/P02-1040/>.
- [12] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision, 2015. URL <https://arxiv.org/abs/1512.00567>.
- [13] Rafael Müller, Simon Kornblith, and Geoffrey Hinton. When does label smoothing help?, 2020. URL <https://arxiv.org/abs/1906.02629>.
- [14] Jörg Tiedemann, Mikko Aulamo, Daria Bakshandaeva, Michele Boggia, Stig-Arne Grönroos, Tommi Nieminen, Alessandro Raganato, Yves Scherrer, Raul Vazquez, and Sami Virpioja. Democratizing neural machine translation with opus-mt, 2023. URL <https://arxiv.org/abs/2212.01936>.